Solidity Basic

Anatoly Krestenko

Vega Institute Foundation

Feb 20, 2026

Plan

- Variables and Types
- Control Structures
- Data Structures
- Data Locations
- Functions and Modifiers
- Events, Error Handling, and Invariants
- Smart Contract Constructor

Almanac

Solidity by Example (v0.8.26) by Cyfrin

Primitive Data Types

- `bool`: true / false
- `uint` stands for unsigned integer, meaning non-negative integers; different sizes are available
- the most commonly used type is `uint256`, which ranges from 0 to $2^{256} - 1$
- negative numbers are allowed for `int` types; like `uint`, different sizes are available from `int8` to `int256`; `int128` ranges from -2^{127} to $2^{127} - 1$
- `address` refers to an EVM account address
- the `bytes` type represents a sequence of bytes: fixed-size byte arrays or dynamically sized byte arrays
- `string` is a dynamically sized UTF-8 string (internally represented as a dynamic byte array)

Types of Variables

- **Local:** declared inside a function and not stored on the blockchain
- **State:** declared outside a function and stored on the blockchain. To write or update a state variable, you must send a transaction. On the other hand, you can read state variables for free, without paying a transaction fee.
- **Global:** provides information about the blockchain and the current execution context

Control Structures

- Most control structures known from curly-brace languages are available in Solidity: `if`, `else`, `while`, `do`, `for`, `break`, `continue`, and `return`, with the usual semantics known from C or JavaScript.
- Solidity supports `for`, `while`, and `do-while` loops; avoid unbounded loops, as they can hit the gas limit and cause your transaction to fail.
- For the reason above, `while` and `do-while` loops are used less frequently in production contracts.

Mapping

A mapping in Solidity is a built-in key–value storage type that associates a key k with a value v, where the storage location of v is computed as $\mathrm{keccak256}(\mathrm{abi.encode}(k, p))$.

$$\mathrm{slot}(k) = \mathrm{keccak256}\left(\mathrm{abi.encode}(k,\ p)\right)$$

Where:

- k — mapping key
- p — base storage slot of the mapping variable
- $\mathrm{abi.encode}(k, p)$ — ABI encoding of k and p, both left-padded to 32 bytes
- $\mathrm{keccak256}(\cdot)$ — Ethereum hash function

Array

An array in Solidity is an ordered collection of elements of the same type, stored contiguously in memory or in storage. Each element is accessed by its zero-based index.

- fixed-size array (storage), where p is the base storage slot of the array variable and i is the zero-based index of the element:

$$\text{slot}(i) = p + i$$

- dynamic array (storage):

$$\text{slot}(p) = \text{length}$$

$$\text{base} = \text{keccak256}(p)$$

$$\text{slot}(i) = \text{keccak256}(p) + i$$

- dynamic array (memory), where b is the base memory pointer of the array and each element occupies 32 bytes:

$$\text{address}(i) = b + 32 + 32 \cdot i$$

Struct

A custom struct is a composite type whose fields are stored sequentially according to standard storage packing rules.

- storage layout: if a struct starts at slot p:

$$\text{slot}(\mathit{field}_i) = p + \text{offset}_i$$

- inside a mapping, struct fields are stored starting from base:

$$\text{base} = \mathrm{keccak256}(k \parallel p)$$

$$\text{slot}(\mathit{field}_i) = \text{base} + \text{offset}_i$$

- in memory there is no packing, and each field occupies 32 bytes

Data Locations

Variables are declared as either `storage`, `memory`, or `calldata` to explicitly specify the location of the data.

- `storage` — the variable is a state variable (stored on the blockchain)
- `memory` — the variable lives in memory and exists only during the function call
- `calldata` — a special data location that contains function arguments (read-only)

Functions

```
function <name>(<params>)
    <visibility>
    <stateMutability>
    <virtualOrOverride>
    <modifiers>
    returns (<returnParams>)
{
    ...
}
```

Visibility and Mutability

- Solidity Cheatsheet
- Getter functions can be declared `view` or `pure`:
  - `view` declares that no state will be changed
  - `pure` declares that no state variables will be read or changed
  - `payable` declares that the function can receive ETH (`msg.value` may be > 0)
  - by default, functions are non-`view`, non-`pure`, and non-`payable`
- Functions and state variables must declare whether they are accessible by other contracts:
  - `public` — any contract or account can call
  - `private` — only within the contract that defines it
  - `internal` — only within the contract and derived contracts (default)
  - `external` — only other contracts and accounts can call

Function Modifiers

Modifiers are code that can be executed before and/or after a function call. Modifiers can be used to:

- manage access
- validate inputs
- guard against reentrancy attacks
- log actions
- manage state (e.g., pausability)

Errors

An error will undo all changes made to the state during a transaction.

- `require` is used to validate inputs and conditions before execution
- `revert` is similar to `require`; it is typically used to abort execution with an error
- `assert` is used to check for conditions that should never be false; a failing assertion usually indicates a bug

Events

Events in Solidity are a powerful tool that enables various advanced functionalities and architectures. Common use cases include:

- event filtering and monitoring for real-time updates and analytics
- event log analysis and decoding for data extraction and processing
- event-driven architectures for decentralized applications (dApps)
- event subscriptions for real-time notifications and updates

Constructor

A constructor is an optional function that is executed once, upon contract creation.

- it can be used to initialize state variables
- it can set privileged roles (e.g., the initial owner)
- it can validate deployment parameters

Exercises

- **Task 1.** Open Remix and implement a `Counter` contract:
  - pass the initial value through the constructor
  - implement `increment()` and `decrement()` functions
  - implement a getter function
  - test different visibility and mutability types
- **Task 2.** In Remix, implement an `Ownable` contract:
  - the deployer must be the initial owner
  - implement a `transferOwnership(address newOwner)` function
  - add an `onlyOwner` modifier
- **Task 3.** Modify your `Counter` contract from Task 1 to inherit from the `Ownable` contract implemented in Task 2. Deploy the updated contract to the Virtual EVM. Test your contract using different addresses.

References I

Ethereum Foundation
Ethereum Virtual Machine.

Cyfrin
Solidity by Example.

Ethereum Foundation
Solidity Documentation.